

Automated Deployment and Operations in an OpenStack Cloud

Hareesh Satyendra Reddy Mullangi

Telecommunication Systems

Blekinge Institute of Technology

Karlskrona, Sweden

haml24@student.bth.se

Abstract—This project focuses on the design and realization of an automated, horizontally scalable web service within a cloud environment. The core objective was to transition away from traditional manual server management by building a layered architecture consisting of a secure entry point, a centralized gateway for traffic distribution, and a flexible cluster of backend nodes. By developing a custom suite of orchestrated scripts, I documented the entire lifecycle of the environment, including the creation of the digital network foundation, the scaling of resources to meet demand, and the eventual teardown of the infrastructure. Performance was analyzed using benchmarking tools to evaluate how the system handles stress as it grows. The results highlight the relationship between hardware expansion and the natural limits of proxy-based systems, specifically identifying the orchestration overhead inherent in a cloud setting.

Index Terms—OpenStack, Ansible, Automation, Nginx, Horizontal Scaling, Apache Benchmark, Flask.

I. INTRODUCTION

In the world of modern cloud computing, being able to set up and manage services quickly is no longer just a luxury; it is a requirement. For this project, I have built a web cluster that is not just a collection of static servers, but a responsive environment managed through specialized deployment and operational scripts. The project is based on the idea that an engineer should be able to launch a complex network of servers without needing to click through hundreds of manual menus.

By merging networking principles with automation, I created a system that handles all the difficult tasks of setting up virtual networks and security layers automatically. This journey involved building the system from nothing, testing it by simulating hundreds of users to see when it would break, and thinking about how this design would need to change if it were used by people all over the world. This report explains the logic behind

my choices and the lessons learned from watching the system perform under pressure.

II. DESIGN MOTIVATION

The motivation for this design was driven by the desire to eliminate the common failures of manual setups: human error, inconsistent configurations, and the inability to react quickly. When a website gets a sudden surge of visitors.

A. Scalability

The most important goal of this project was scalability. In many old-fashioned setups, when a server gets too busy, the only answer is to buy a bigger, more expensive server. I wanted to move away from that. My design uses horizontal scaling, which means instead of making one server bigger, we just add more small worker servers to the group. This is a much more flexible way to work. If the website gets busy, we launch more workers; when the traffic goes away, we turn them off. This makes the infrastructure feel less like a rigid building and more like a team that can grow or shrink as needed.

B. Infrastructure as Code

A primary motivation for this project was the implementation of Infrastructure as Code. I wanted the entire environment to be defined by logic and scripts rather than manual interactions with a dashboard. By treating the setup as code, the infrastructure becomes documented and repeatable. This means that if the environment needs to be moved to a different project or provider, the underlying logic remains consistent. It allows for version control over the hardware setup, ensuring that any changes to the network or server configurations are tracked and can be easily rolled back if necessary.

C. Efficiency

I wanted to ensure that every second spent on the project was used effectively. Configuring five or ten servers by hand is not just boring, it is an invitation for mistakes. By using automation, I made the process incredibly efficient. Once the logic was written, it did not matter if I was launching two nodes or five the effort on my part stayed the same. This efficiency allows an engineer to focus on the big picture of the application rather than the small, repetitive details of server settings.

D. Practical Automation

Using Ansible and custom scripts allowed me to create a standardized baseline for my servers. Every time a new server is added to the cluster, the automation ensures it is a perfect copy of the ones already running. This consistency is vital for a web service. If one server is set up slightly differently than the others, it can cause bugs that are almost impossible to find. Automation removes that risk by making the setup process documented, repeatable, and fast.

III. INSTALL STAGE (DEPLOYMENT)

The deployment stage is where the vision of the project becomes a reality. I designed this process to be a hands-off experience where the scripts do the heavy lifting of building a digital world from scratch.

A. Establishing the Network Foundation

It all starts with building the foundation. The process begins by creating a private, secure virtual environment in the cloud where our servers can live. This area is isolated from the rest of the internet to keep it safe. To allow our servers to actually communicate with the outside world, I had to build a virtual router that connects our private space to the public provider network. This ensures that while our internal workers are tucked away safely, they can still receive instructions and send out data when needed.

B. Security and Access Control

Once the virtual network is ready, I set up a multi-layered security protocol. I created specific security group rules that act as filters to decide exactly who is allowed to talk to our servers. I configured the system so that the general public can only reach the front-end load balancer gateway, while the worker nodes stay completely hidden within the internal network. For my own administrative work, I created a separate, highly secure bastion host that acts as the only way to get inside the system for maintenance.

C. Service Provisioning and Configuration

With the network and security ready, the script begins the process of provisioning the actual servers. It launches the front-end gateway first, which acts as the entry point for all incoming traffic. Then, it starts the worker nodes. As these servers power on, the automation automatically steps in to install the web software, set up the application code, and start the service. The final step of the deployment involves defining the internal network locations of each new worker within the gateway configuration to ensure the system is synchronized and ready to perform. This creates a fully linked system where every instance is synchronized and ready to perform its specific role the moment it is active.

IV. OPERATIONS STAGE

In the operations phase, the system actually lives and breathes. This is where we handle the day-to-day changes that a real-world service faces. The main feature of this mode is the ability to change the size of the cluster on the fly.

If I decide that the cluster needs to grow to handle more users, I do not have to delete everything and start over. Instead, I simply update a configuration file with the new number of nodes I want. The operations script then compares what is currently happening in the cloud with what I have requested. If it sees a gap, it launches the new nodes and configures them just like the original ones. The operational logic ensures that new nodes are added to the distribution list so that they can take their share of the work. It adds the new nodes to the distribution list so that they can start taking their share of the work immediately. This makes the transition from a small cluster to a large one feel smooth and seamless.

V. CLEANUP STAGE

In a cloud environment, leaving unused servers running is wasteful and expensive. I built a cleanup mode that is just as thorough as the installation. It runs the entire process in reverse, like a movie playing backward. It starts by gracefully shutting down and removing the worker nodes and the gateway. Once the servers are gone, it clears out the security filters and tears down the virtual router. Finally, it deletes the private network itself. This ensures that the cloud project is left exactly as I found it, with no lingering resources or ghost settings taking up space.

VI. STATISTICAL ANALYSIS

To prove that the system actually works as intended, I ran a series of performance tests. I wanted to see if adding more servers actually made the website faster or if there was a limit to how much we could scale.

A. Experimental Setup

I used a benchmarking tool to simulate hundreds of people trying to access the website at the exact same time. I ran this test five times once for a single node, then two, and so on up to five nodes. To make sure the data was honest, I did not just take the first result I got. Initial testing of the four-node configuration exhibited experimental anomalies due to transient network latency. To ensure data integrity, the test was re-conducted under stable conditions to reflect true system performance

B. Functional Validation

Before conducting the stress tests, I performed a functional verification to ensure that the gateway was correctly distributing traffic. As shown in Fig. 1, successive `curl` requests directed at the public IP were answered by different internal worker nodes in a perfect Round-Robin sequence. This confirms that the gateway configuration was correctly mapped to the internal IP addresses of the worker pool, proving that the load-balancing logic was operational before the performance evaluation began.

```
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:03 Serving from final-auto-node2 (10.0.0.165)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:06 Serving from final-auto-node3 (10.0.0.89)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:08 Serving from final-auto-node4 (10.0.0.187)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:10 Serving from final-auto-node5 (10.0.0.193)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:12 Serving from final-auto-node1 (10.0.0.88)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:13 Serving from final-auto-node2 (10.0.0.165)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:14 Serving from final-auto-node3 (10.0.0.89)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:15 Serving from final-auto-node4 (10.0.0.187)
hareesh_18@Hareesh:~/NSQ_Project_Final$ curl http://194.47.155.119/
15:02:16 Serving from final-auto-node5 (10.0.0.193)
```

Fig. 1. Verification of load balancing across five active nodes.

C. Collected Results

D. Performance Analysis by Stage

1) *1-Node Stage*: This is our baseline. With a single worker, the proxy has no load-balancing logic to compute. As seen in Fig. 2, this stage provided the highest

TABLE I
SYSTEM PERFORMANCE METRICS (N=1000, C=10)

Nodes	Req/Sec	Mean RTA (ms)	SD (ms)	Median (ms)
1	182.36	54.84	94.2	44
2	181.93	54.97	92.0	44
3	177.39	56.37	95.1	44
4	175.64	56.93	94.9	45
5	172.65	57.92	90.6	45

throughput because the path between the user and the code was at its shortest and simplest.

```
Requests per second: 182.36 [#/sec] (mean)
Time per request: 54.836 [ms] (mean)
Time per request: 5.484 [ms] (mean, across all concurrent requests)
Transfer rate: 38.82 [Kbytes/sec] received

Connection Times (ms)
min mean[+/-sd] median max
Connect: 19 29 94.1 20 1094
Processing: 21 25 3.4 24 63
Waiting: 21 25 3.4 24 63
Total: 41 54 94.2 44 1117
```

Fig. 2. Benchmarking for 1 worker node.

2) *2-Node Stage*: Adding a second node shows almost identical speed. This proves the architecture can handle the transition from a single server to a cluster without adding any noticeable delay or "friction" to the user experience.

```
Requests per second: 181.93 [#/sec] (mean)
Time per request: 54.967 [ms] (mean)
Time per request: 5.497 [ms] (mean, across all concurrent requests)
Transfer rate: 38.64 [Kbytes/sec] received

Connection Times (ms)
min mean[+/-sd] median max
Connect: 18 29 92.1 20 1086
Processing: 21 25 3.3 24 57
Waiting: 21 25 3.3 24 57
Total: 40 54 92.0 44 1110
```

Fig. 3. Benchmarking for 2 worker nodes.

3) *3-Node and 4-Node Stages*: As we grew to three and four nodes, we started to see a slight dip in throughput. This is an important observation; it shows that while we have more workers, the gateway is spending more energy coordinating the traffic, which adds a tiny amount of overhead to each request.

Requests per second:	177.39	[/sec]	(mean)
Time per request:	56.372	[ms]	(mean)
Time per request:	5.637	[ms]	(mean, across all concurrent requests)
Transfer rate:	37.68	[Kbytes/sec]	received
Connection Times (ms)			
	min	mean[+/-sd]	median max
Connect:	19	30 94.5	20 1084
Processing:	22	25 10.7	24 269
Waiting:	21	25 10.7	24 269
Total:	41	54 95.1	44 1109

Fig. 4. Benchmarking for 3 worker nodes.

Requests per second:	175.64	[/sec]	(mean)
Time per request:	56.934	[ms]	(mean)
Time per request:	5.693	[ms]	(mean, across all concurrent requests)
Transfer rate:	37.31	[Kbytes/sec]	received
Connection Times (ms)			
	min	mean[+/-sd]	median max
Connect:	19	30 94.5	20 1096
Processing:	22	26 5.9	24 97
Waiting:	21	25 5.9	24 97
Total:	41	55 94.9	45 1133

Fig. 5. Benchmarking for 4 worker nodes.

4) *5-Node Stage*: At the five-node stage, the orchestration overhead—often conceptualized as a ‘Proxy Management Tax’—became most apparent. The throughput reached its lowest point of 172.65 RPS, illustrating that the cluster’s efficiency is ultimately constrained by the hardware limitations of the gateway tier.

Requests per second:	172.65	[/sec]	(mean)
Time per request:	57.922	[ms]	(mean)
Time per request:	5.792	[ms]	(mean, across all concurrent requests)
Transfer rate:	36.69	[Kbytes/sec]	received
Connection Times (ms)			
	min	mean[+/-sd]	median max
Connect:	19	30 90.3	20 1081
Processing:	22	26 5.8	24 67
Waiting:	22	26 5.6	24 66
Total:	41	56 90.6	45 1108

Fig. 6. Benchmarking for 5 worker nodes.

E. Overall Analysis

This data indicates that the front-end gateway functioned as a computational bottleneck. The gateway was required to dedicate significant resources to coordinating traffic and maintaining connection states, a factor that eventually outweighed the additional processing power provided by the backend worker nodes. Consequently, the results show that the automation is technically robust, but horizontal scaling of the backend effectively requires a more powerful gateway to handle the increased coordination load.

VII. LARGE SCALE REASONING

When we think about taking this project and scaling it up for thousands or millions of users, the problems we

face become much more theoretical and complex.

A. Potential Problems at Scale

One of the biggest issues at a massive scale is internal noise. In my project, five servers talking to each other is quiet. But if you had a thousand servers, the sheer amount of background communication just to check if everyone is alive would start to slow down the actual network. Another problem is the limitation of the central gateway. At a certain point, a single entry point no matter how powerful cannot handle the load. The system would start to experience queuing, where users have to wait in a digital line just to talk to the gateway. Additionally, our automation would need to change. Checking a thousand servers one-by-one with a script is too slow. We would need a system where the servers announce themselves when they are ready, rather than the script having to go find them.

B. Multi-Location Challenges

If this service had to work for people in different parts of the world, like Europe and Asia, we would face the speed of light problem. No matter how fast our servers are, data can only travel through fiber optic cables so quickly. A user in Asia should not have to wait for their data to travel to a server in Europe and back. To fix this, we would need to deploy mini-clusters in every region. However, this creates a massive headache for data. If a user in Europe changes their password, that change needs to show up in the Asia cluster immediately. Keeping data perfectly synchronized across the world while keeping the service fast is one of the hardest problems in computer science. If you wait for all servers to agree on the data, the service feels slow. If you do not wait, different users might see different information. This tradeoff between fresh data and fast performance is a constant battle when scaling globally.

VIII. CONCLUSION

This project successfully demonstrated how cloud systems can be effectively managed using orchestrated scripts. I was able to build, scale, and destroy a web cluster using nothing but logic and scripts. Even though the performance testing showed that my gateway had a physical limit, the experiment was a success because it revealed exactly where the system would need to be improved for a real-world launch. The automation worked perfectly, reducing the time it takes to grow the cluster to just a little over a minute. This project provided a deep understanding of how modern cloud companies manage their massive networks of servers every day.